

NLP202 Assignment 1

Shriya Sravani Y
UCSC
sy4@ucsc.edu

Abstract

This report presents two machine learning models, Logistic Regression and LSTM, for sentiment analysis on the IMDB dataset. The objective is to classify movie reviews as positive or negative. Key steps include preprocessing the dataset, implementing models, hyperparameter tuning, and evaluating the models' performance. Observations and insights into single-instance and minibatched versions are also discussed, alongside answers to deliverables. This report presents the results of sentiment analysis using logistic regression on the IMDB dataset.

1 Introduction

Sentiment analysis is a fundamental task in natural language processing (NLP), aiming to classify text based on sentiment. This assignment focuses on comparing Logistic Regression and LSTM models. The Logistic Regression model serves as a baseline with an embedding layer and a linear output layer, while the LSTM model incorporates sequence modeling through recurrent layers.

Key objectives include:

1. Implementing both models with padding and masking for minibatching.
2. Evaluating single-instance and minibatched versions for correctness.
3. Hyperparameter tuning for batch sizes, learning rates, dropout rates, and epochs.

1.1 Dataset description

The IMDB dataset consists of 50,000 movie reviews labeled as positive or negative, divided into training and test sets with 25,000 reviews each. Reviews vary in length, necessitating padding for minibatching. The dataset split is given as follows:

Training set: 20,000 reviews

Validation set: 5,000 reviews

Test set: 25,000 reviews

1.2 Padding and masking

To handle variable-length input sequences, padding and masking are applied:

1. Padding: Shorter sequences are padded with a special token (`PAD_IDX = 0`) to match the maximum sequence length in the batch.

2. Packing and Unpacking:

Sequences are packed using `pack_padded_sequence` to ignore padding during LSTM computation.

Outputs are unpacked using `pad_packed_sequence` for further processing

2 Building a Logistic Regression model

The Logistic Regression model consists of:

- Embedding Layer: Converts input tokens to dense vector representations.

- Linear Output Layer: Maps the averaged embedding to a single scalar for binary classification.

Here we will be comparing the model's performance for single-instance and minibatching versions.

Single-Instance: Processes one review at a time, ensuring precise loss computation.

Minibatching: Processes multiple reviews simultaneously, requiring padding for uniform lengths.

Observation: Minibatching results in comparable loss values to single-instance processing due to fixed random seed initialization.

2.1 Hyperparameter Tuning

Experiments were conducted with varying batch sizes (1, 16, 32, 64, 128), learning rates (1e-3, 1e-4), dropout rates (0.3, 0.5), and epochs (10, 20). Key results are summarized below:

Batch Sizes	Learning Rates
1, 16, 32, 64, 128	0.001, 0.0001

Table 1: Hyperparameters used for training the Logistic Regression model.

The model was run for 10 and 20 epochs. The optimization function used is Adam and loss function is BCEWithLogitsLoss.

Batch Size	LR	Train Time (s)	Validation Acc.
1	0.001	2066.31	89.22%
16	0.001	120.11	88.94%
32	0.001	44.76	88.12%
64	0.001	30.95	87.10%
128	0.001	24.73	84.48%

Table 2: Training time and validation/test accuracy for different batch sizes and learning rates.

Batch Size	LR	Train Time	Validation Acc.
1	0.0001	2087.63	88.50%
16	0.0001	123.84	88.02%
32	0.0001	46.12	87.35%
64	0.0001	32.65	86.50%
128	0.0001	25.43	83.80%

Table 3: Training time and validation/test accuracy for different batch sizes and learning rate 0.0001.

2.2 Observations

- The minibatching scores are approximately the same as the single-instance model due to deterministic initialization and consistent padding/masking across batches. Minor differences occur due to rounding errors and accumulation of gradient updates over a batch.
- Larger batch sizes reduce training time significantly because more data is processed in parallel, minimizing overhead per epoch.
- A learning rate of 0.001 outperforms 0.0001, yielding higher validation accuracy across all batch sizes.
- Best model on the dev and test sets after training is:
 - Best Model Parameters: Batch Size = 1, Learning Rate = 0.001, Dropout = 0.3, Epochs = 20
 - Validation Accuracy: 89.22%
 - Test Accuracy: 83.34%
- The model occasionally misclassifies reviews with ambiguous sentiment or mixed opinions, highlighting its limitation in capturing subtle contextual cues.

3 Building LSTM feature extractor model

For the LSTM model,

Batch Size	Learning Rate
16, 32, 64	0.001, 0.0001

Table 4: Hyperparameters used for training the LSTM model.

The model was run for 10 and 20 epochs. The optimization function used is Adam and loss function is BCEWithLogitsLoss.

Batch Size	Training Time	Validation Acc.	LR
16	5071.27	87.92%	0.001
32	633.43	88.10%	0.001
64	389.52	82.72%	0.001
16	610.76	86.32%	0.0001
32	649.99	87.34%	0.0001
64	381.18	85.80%	0.0001

Table 5: Comparison of training time and validation accuracy for different batch sizes and learning rates.

3.1 Observations

- The minibatching output scores are not exactly the same due to the averaging of gradients across multiple instances within a batch. However, the difference is negligible if the dataset is shuffled well, and the same random seed ensures consistent initialization.
- For batch size 32, a learning rate of 0.001 yielded the best validation accuracy of 88.10
- Accuracy of Best Model on Dev and Test Sets:
 - Batch Size: 32, LR : 0.001, epochs : 20
 - Validation Accuracy: 88.10Test Accuracy: 84.63
- The test set evaluation: Test Loss: 0.7860, Test Accuracy: 0.8463.

4 Appendix

Logistic Regression Plots:

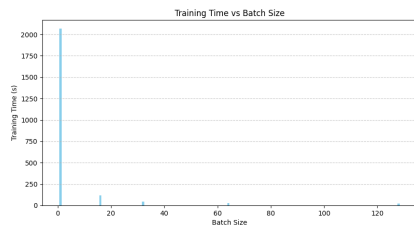


Figure 1: Training Time vs Batch Size

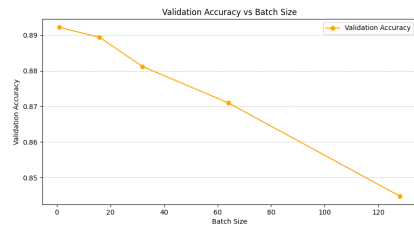


Figure 2: Training Time vs Batch Size

LSTM Plots:

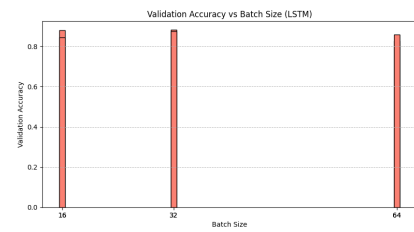


Figure 3: Training Time vs Batch Size

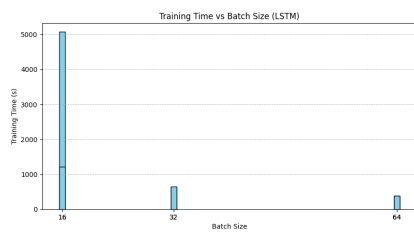


Figure 4: Training Time vs Batch Size